Purpose of This Challenge

This challenge evaluates how you design, reason about, and implement a real-world system using modern full-stack and backend concepts. There is no single correct solution. We are interested in your architectural thinking, API design, use of asynchronous processing, and how clearly you communicate tradeoffs and assumptions.

Problem Statement

Design and implement a simple CRM (Customer Relationship Management) system that supports managing customer-related data, multiple companies organized as parent and child companies, access control based on user roles, and at least one asynchronous workflow using a queue system.

Required Technologies

Backend: Python, Django, GraphQL

Frontend: React, Apollo Client (or equivalent)

Architecture / Infrastructure: Nx workspace, GraphQL Federation, Kafka, Docker & Docker Compose

High-Level Architectural Expectations

The system should be composed of multiple services rather than a single monolith. Each backend service must expose a GraphQL API, and a GraphQL Gateway must federate these APIs. The frontend must communicate only with the gateway. Kafka must be used for asynchronous processing. All services must be runnable together using Docker Compose.

CRM Domain (Guidance, Not a Specification)

Your CRM should support a basic customer lifecycle. You are free to choose your own entities and workflows. Common examples include leads, contacts, deals, and activities. A small, well-designed subset is preferred.

Queue / Asynchronous Workflow Requirement

Your system must include at least one workflow that is processed asynchronously using Kafka. A user action should publish an event, one or more consumers should process it, and the system should update state asynchronously. The UI should reflect eventual consistency.

Queue Design Expectations

Your queue implementation should consider duplicate message delivery, retries, failure handling, and observability. You may choose your own event structure, topics and retry strategy.

Parent & Child Companies

The CRM must support multiple companies organized into a parent and child hierarchy. CRM data should belong to child companies, parent companies may have visibility across their children, and child companies should be isolated.

Access Levels & Permissions

The system should support different access levels. Access control must be enforced server-side and not rely solely on frontend restrictions.

GraphQL & Federation Expectations

We are looking for clean and understandable GraphQL schemas, clear ownership of data per service, and meaningful use of GraphQL Federation without unnecessary complexity.

Frontend Expectations

The frontend should use Apollo Client, communicate only with the GraphQL gateway, handle loading and error states, and clearly reflect asynchronous behavior. UI polish is not a priority.

Testing Expectations

Exhaustive test coverage is not required. However, some tests should exist around important logic, or you should clearly explain what you would test next.

Docker Compose (Mandatory)

Your submission must include a working Docker Compose setup that runs the entire system end-to-end using a single 'docker compose up --build' command.

README Requirements

Your README must explain how to run the project, list service URLs, describe the architecture, explain the asynchronous workflow, and document assumptions and tradeoffs.

Deliverables

Please submit a Git repository containing your source code, a working docker-compose.yml file, and a clear README.

What We Evaluate

We evaluate ease of running the system, architectural clarity, meaningful use of Kafka, correct access boundaries, proper use of GraphQL Federation, code quality, and engineering judgment.